

Шпаргалка по современному C++

Многопоточность, шаблоны, модели памяти

Подготовка к зачёту

Содержание

1	Идеальная передача и вариативные шаблоны	3
1.1	lvalue-выражения и rvalue-выражения	3
1.2	lvalue-ссылки и rvalue-ссылки	3
1.3	Универсальные ссылки (forwarding references)	4
1.4	Типы передачи объекта в функцию	4
1.5	Идеальная передача (perfect forwarding)	4
1.6	Вариативные шаблоны (variadic templates)	4
2	Основы многопоточного программирования. Потоки.	5
2.1	Параллелизм и конкурентность	5
2.2	Класс std::thread	5
2.3	Возврат данных из потока	5
2.4	Другое	6
3	Мьютексы	6
3.1	Состояние гонки (race condition)	6
3.2	Стандартный мьютекс std::mutex	6
3.3	std::lock_guard и std::unique_lock	6
3.4	Взаимоблокировка (deadlock)	6
3.5	Защита разделяемых данных во время инициализации	6
4	Механизмы синхронизации	7
4.1	Условные переменные (std::condition_variable)	7
4.2	Асинхронные задачи: std::async, std::future	7
4.3	std::packaged_task	7
4.4	std::promise	7
5	Потокобезопасные стек и очередь с блокировками	8
5.1	Потокобезопасная структура данных	8
5.2	Потокобезопасный стек с блокировками (на основе std::stack)	8
5.3	Потокобезопасная очередь с блокировками	9
5.4	Очередь на основе односвязного списка с двумя мьютексами	9
6	Модели памяти	9
6.1	Барьеры памяти и упорядочение	9
6.2	Модели памяти в C++	9
6.3	Атомарные типы и операции	10

7	Потокобезопасные стек и очередь без блокировок	10
7.1	Основные определения	10
7.2	Реализация lock-free стека	10
7.3	Управление памятью (Hazard pointers)	11
7.4	Реализация lock-free очереди	11
8	Пул потоков	11
8.1	Что такое пул потоков?	11
8.2	Реализация пула потоков на C++	11
8.3	Предотвращение конкуренции за очередь работ	12

1 Идеальная передача и вариативные шаблоны

1.1 lvalue-выражения и rvalue-выражения

Выражение — последовательность операторов и операндов, задающая вычисление. У каждого выражения есть тип (например, `int`) и **категория значения** (value category).

- **lvalue** (location value) — выражение, обозначающее объект, имеющий адрес в памяти (который можно взять `&`). Примеры: имя переменной, оператор разыменования `*ptr`, строковый литерал (кроме массивов). lvalue можно ставить слева от присваивания (если не константное).
- **rvalue** (read value) — выражение, не имеющее устойчивого адреса (временные объекты, литералы, результат функции, возвращающей не ссылку). Не может стоять слева от присваивания.

Примеры:

```
int x = 5;           // x - lvalue, 5 - rvalue
int* p = &x;        // &x - rvalue (address as temporary)
int y = x + 1;      // x+1 - rvalue
++x;                // prefix ++ returns lvalue
x++;                // postfix ++ returns rvalue
```

1.2 lvalue-ссылки и rvalue-ссылки

- **lvalue-ссылка** (`T&`) связывается только с lvalue (можно с `const lvalue` и с `rvalue`, если ссылка `const`, но временный объект живёт до конца выражения).
- **rvalue-ссылка** (`T&&`) связывается только с rvalue (или с `xvalue`). Позволяет реализовать перемещение.

Разделение нужно для оптимизации: из `rvalue` можно «украсть» ресурсы, так как они всё равно будут уничтожены. Перегрузка по категории:

```
void foo(int&) { std::cout << "lvalue\n"; }
void foo(int&&) { std::cout << "rvalue\n"; }
int main() {
    int a = 10;
    foo(a);    // lvalue
    foo(20);   // rvalue
}
```

Если параметр — `T&&` (и это не шаблон), то он связывается только с `rvalue`. Однако идентификатор этого параметра сам является lvalue внутри функции:

```
void bar(int&& x) {
    foo(x);    // calls foo(int&), because x is a named object (lvalue)
}
```

Чтобы передать его как `rvalue`, нужно использовать `std::move`.

`std::move` — это просто `static_cast<T&&>(value)`. Он не перемещает сам, а только приводит тип к `rvalue-ссылке`, разрешая перемещение.

1.3 Универсальные ссылки (forwarding references)

Если в шаблоне параметр имеет вид `T&&`, где `T` — шаблонный параметр этой функции, то это **универсальная ссылка**. Правила вывода:

- Если передан lvalue типа `U`, то `T` выводится как `U&`, и свёртка даёт `U& && → U&` (lvalue-ссылка).
- Если передан rvalue, то `T` выводится как `U`, и получается `U&&`.

Таким образом, универсальная ссылка может принять что угодно и сохранить категорию (нужно использовать `std::forward` для проброса).

1.4 Типы передачи объекта в функцию

1. По значению копированием (`T arg`) — копия, безопасно, но дорого для больших объектов.
2. По значению перемещением — если передать rvalue, будет перемещение. Полезно для фабрик.
3. По lvalue-ссылке (`T&`) — без копирования, можно изменять. Нельзя передать rvalue.
4. По rvalue-ссылке (`T&&`) — только для rvalue, обычно для перемещения.
5. По константной lvalue-ссылке (`const T&`) — принимает всё (и lvalue, и rvalue), без копирования, но нельзя изменять.
6. По универсальной ссылке (`T&&` в шаблоне) — сохраняет категорию, идеальная передача с `std::forward`.

1.5 Идеальная передача (perfect forwarding)

`std::forward<T>(arg)` условно приводит `arg` к rvalue, если `T` выведен как не ссылочный тип (т.е. исходный аргумент был rvalue). Реализация:

```
template<typename T>
T&& forward(typename std::remove_reference<T>::type& t) noexcept {
    return static_cast<T&&>(t);
}
```

Отличие от `std::move`: `move` всегда даёт rvalue, а `forward` — только если исходный аргумент был rvalue.

1.6 Вариативные шаблоны (variadic templates)

Позволяют принимать произвольное количество аргументов. Пакет параметров обозначается `...`.

```
template<typename... Args>
void print(Args... args) {
    (std::cout << ... << args); // fold expression (C++17)
}
```

Раскрытие пакета возможно во многих контекстах: вызов функции, инициализация, списки базовых классов, using-декларации.

Свёртка (fold expressions):

- (pack op ...) — левоассоциативная
- (... op pack) — правоассоциативная

`sizeof...(Args)` — количество аргументов в пакете.

Совместно с идеальной передачей:

```
template<typename T, typename... Args>
std::unique_ptr<T> make_unique(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}
```

2 Основы многопоточного программирования. Потоки.

2.1 Параллелизм и конкурентность

- **Конкурентность** (concurrency) — логическая одновременность выполнения нескольких задач (могут чередоваться на одном ядре).
- **Параллелизм** (parallelism) — физическое одновременное выполнение на нескольких ядрах.

Процесс — экземпляр программы с собственным адресным пространством. **Поток** — легковесная единица выполнения внутри процесса. Преимущества потоков: разделение памяти (легче обмениваться данными), меньше накладных расходов на создание. Недостатки: сложнее синхронизация, возможны гонки данных.

2.2 Класс `std::thread`

Создание потока:

```
void func(int x) { ... }
std::thread t(func, 42);
t.join(); // wait for completion
t.detach(); // detach (thread continues, std::thread no longer owns it)
```

Если поток не был `join` или `detach` до разрушения объекта, вызывается `std::terminate`. При исключении в потоке, который создаёт другой поток, нужно гарантировать вызов `join()`, обычно через RAII.

2.3 Возврат данных из потока

Через глобальную переменную, но нужна синхронизация. Можно использовать `std::ref` для оборачивания ссылки:

```
void compute(int& result) { result = 42; }
int main() {
    int res = 0;
    std::thread t(compute, std::ref(res));
    t.join();
    // res == 42
}
```

Почему нельзя обычную ссылку? `std::thread` копирует аргументы, а ссылка не копируется. `std::reference_wrapper` — объект, который ведёт себя как ссылка (можно копировать). `std::ref` создаёт его.

2.4 Другое

Идентификация потока: `std::this_thread::get_id()`, `std::thread::id`. Перемещение владения: `std::thread t1(f); std::thread t2 = std::move(t1);`. Создание многих потоков:

```
std::vector<std::thread> threads;
for (int i = 0; i < 8; ++i) threads.emplace_back(func, i);
for (auto& t : threads) t.join();
```

`std::mem_fn` оборачивает указатель на метод в функциональный объект.

3 Мьютексы

3.1 Состояние гонки (race condition)

Разделяемые данные — данные, доступные нескольким потокам. **Состояние гонки** — ситуация, когда результат зависит от непредсказуемого порядка выполнения потоков. **Гонка данных** (data race) — разновидность состояния гонки, когда два потока одновременно обращаются к одной памяти, хотя бы один пишет, без синхронизации. В C++ это приводит к UB (разрушению памяти, непредсказуемым значениям).

3.2 Стандартный мьютекс `std::mutex`

Методы: `lock()` (блокирует, если занят — ждёт), `try_lock()` (не блокирует, возвращает `bool`), `unlock()`. Необходимо всегда разблокировать, используя RAII.

3.3 `std::lock_guard` и `std::unique_lock`

`std::lock_guard` — простой RAII: захватывает мьютекс в конструкторе, освобождает в деструкторе. Нельзя переместить или досрочно освободить.

`std::unique_lock` более гибкий: поддерживает отложенную блокировку, раннее освобождение, перемещение. Используется с `std::condition_variable`. Недостаток: небольшой оверхед.

3.4 Взаимоблокировка (deadlock)

Возникает, когда два потока удерживают мьютексы и ждут друг друга. Решение: захватывать мьютексы в одинаковом порядке или использовать `std::lock` (захват нескольких мьютексов атомарно):

```
std::lock(mtx1, mtx2);
std::lock_guard<std::mutex> lk1(mtx1, std::adopt_lock);
std::lock_guard<std::mutex> lk2(mtx2, std::adopt_lock);
```

или `std::scoped_lock` (C++17).

3.5 Защита разделяемых данных во время инициализации

Паттерн Double-Checked Locking — небезопасен в C++ (может быть UB). Вместо этого `std::once_flag` и `std::call_once`:

```
std::once_flag flag;
std::call_once(flag, []{ /* initialization */ });
```

4 Механизмы синхронизации

4.1 Условные переменные (`std::condition_variable`)

Позволяет потоку ждать наступления условия. Используется вместе с `std::unique_lock<std::mutex>`.

```
std::mutex mtx;
std::condition_variable cv;
bool ready = false;

void worker() {
    std::unique_lock<std::mutex> lk(mtx);
    cv.wait(lk, []{ return ready; }); // spurious wakeups handled by
    predicate
    // work
}

void set_ready() {
    std::lock_guard<std::mutex> lk(mtx);
    ready = true;
    cv.notify_one(); // or notify_all()
}
```

Ложные пробуждения (spurious wake) — поток может проснуться без уведомления, поэтому нужен цикл проверки.

4.2 Асинхронные задачи: `std::async`, `std::future`

`std::async` запускает функцию асинхронно и возвращает `std::future`, из которого можно получить результат.

```
int f(int x) { return x*x; }
std::future<int> fut = std::async(f, 5);
int res = fut.get(); // blocks until result is ready
```

Политики запуска: `std::launch::async` (новый поток), `std::launch::deferred` (ленивый вызов при `get()`).

4.3 `std::packaged_task`

Оборачивает вызываемый объект, позволяет получить `std::future` для результата. Можно передавать в другой поток.

```
std::packaged_task<int(int)> task(f);
std::future<int> fut = task.get_future();
std::thread t(std::move(task), 5);
t.join();
int res = fut.get();
```

4.4 `std::promise`

Позволяет явно установить значение в `std::future`. Используется для одноразовой передачи.

```
std::promise<int> prom;
std::future<int> fut = prom.get_future();
std::thread t([&prom]{ prom.set_value(42); });
```

```
t.join();
int res = fut.get();
```

Можно также устанавливать исключение: `prom.set_exception(std::make_exception_ptr(std::run`

5 Потокобезопасные стек и очередь с блокировками

5.1 Потокобезопасная структура данных

Обеспечивает корректный доступ из нескольких потоков (синхронизирует операции). Стандартные контейнеры STL (`vector`, `deque`, `stack`) **не** потокобезопасны. `std::stack` имеет недостатки: методы `top()` и `pop()` разделены, что приводит к `race condition`. Поэтому в стандартной библиотеке они так и оставлены — пользователь сам должен обеспечить синхронизацию.

5.2 Потокобезопасный стек с блокировками (на основе `std::stack`)

Реализуем методы `push`, `pop` (возвращает значение), `try_pop` (возвращает `bool`). Используем `std::mutex` и `std::lock_guard`.

```
template<typename T>
class threadsafe_stack {
    std::stack<T> data;
    mutable std::mutex m;
public:
    threadsafe_stack() = default;
    void push(T value) {
        std::lock_guard<std::mutex> lock(m);
        data.push(std::move(value));
    }
    std::shared_ptr<T> pop() {
        std::lock_guard<std::mutex> lock(m);
        if (data.empty()) throw std::logic_error("empty");
        std::shared_ptr<T> res(std::make_shared<T>(std::move(data.top())
        ));
        data.pop();
        return res;
    }
    bool try_pop(T& value) {
        std::lock_guard<std::mutex> lock(m);
        if (data.empty()) return false;
        value = std::move(data.top());
        data.pop();
        return true;
    }
    bool empty() const {
        std::lock_guard<std::mutex> lock(m);
        return data.empty();
    }
};
```

Безопасность относительно исключений: при копировании значения может выброситься исключение, но мы используем перемещение.

5.3 Потокобезопасная очередь с блокировками

С поддержкой `wait_and_pop` (ждёт, пока не появится элемент) с использованием `std::condition_variable`

```
template<typename T>
class threadsafe_queue {
    std::queue<T> data;
    mutable std::mutex m;
    std::condition_variable cv;
public:
    void push(T value) {
        std::lock_guard<std::mutex> lock(m);
        data.push(std::move(value));
        cv.notify_one();
    }
    void wait_and_pop(T& value) {
        std::unique_lock<std::mutex> lock(m);
        cv.wait(lock, [this]{ return !data.empty(); });
        value = std::move(data.front());
        data.pop();
    }
    bool try_pop(T& value) {
        std::lock_guard<std::mutex> lock(m);
        if (data.empty()) return false;
        value = std::move(data.front());
        data.pop();
        return true;
    }
};
```

5.4 Очередь на основе односвязного списка с двумя мьютексами

Для повышения производительности можно защищать голову и хвост разными мьютексами. Сложная реализация (здесь опущена для краткости).

6 Модели памяти

6.1 Барьеры памяти и упорядочение

При гонке данных процессор и компилятор могут переупорядочить инструкции. Барьеры памяти — инструкции, запрещающие переупорядочивание. Типы:

- **LoadLoad** — все чтения до барьера завершаются перед чтениями после.
- **LoadStore, StoreLoad, StoreStore**.

Acquire барьер: все чтения/записи после барьера не могут переставить до. **Release** барьер: все чтения/записи до не могут переставить после.

6.2 Модели памяти в C++

В C++ определены атомарные операции с явным указанием порядка:

```
std::atomic<int> x;
x.store(1, std::memory_order_release);
int y = x.load(std::memory_order_acquire);
```

Упорядочения:

- `memory_order_relaxed` — никаких гарантий, только атомарность.
- `memory_order_acquire` — последующие операции не переставятся до этой загрузки.
- `memory_order_release` — предыдущие операции не переставятся после этой записи.
- `memory_order_acq_rel` — объединяет `acquire` и `release`.
- `memory_order_seq_cst` — последовательная согласованность (по умолчанию), глобальный порядок.

Функция `std::atomic_thread_fence` — барьер, независимый от атомарной переменной.

6.3 Атомарные типы и операции

`std::atomic<T>` обеспечивает атомарные операции для тривиально копируемых типов. `std::atomic_flag` — простейший атомарный флаг (методы `clear()`, `test_and_set()`). Реализация спинлока:

```
class spinlock {
    std::atomic_flag flag = ATOMIC_FLAG_INIT;
public:
    void lock() { while (flag.test_and_set(std::memory_order_acquire)); }
    void unlock() { flag.clear(std::memory_order_release); }
};
```

Атомарные `load` и `store` могут использовать разные модели памяти. `compare_exchange_strong/weak` — CAS-операция.

7 Потокобезопасные стек и очередь без блокировок

7.1 Основные определения

- **Неблокирующая структура данных** — хотя бы один поток всегда может сделать прогресс (нет взаимоблокировок).
- **Lock-free** — если один поток приостановлен, другие продолжают.
- **Wait-free** — каждый поток завершит операцию за конечное число шагов.

7.2 Реализация lock-free стека

На основе односвязного списка и CAS (compare-and-swap). Операция `push`:

```
template<typename T>
class lockfree_stack {
    struct node { T data; node* next; };
    std::atomic<node*> head;
public:
```

```

void push(T value) {
    node* new_node = new node{std::move(value), head.load()};
    while (!head.compare_exchange_weak(new_node->next, new_node));
}
};

```

Проблема: утечки памяти при pop. Используются методы подсчёта ссылок или hazard pointers.

7.3 Управление памятью (Hazard pointers)

Каждый поток перед обращением к узлу объявляет указатель «опасным» (hazard pointer). Другие потоки не удаляют узлы, на которые есть hazard pointers. Недостаток: сложность, накладные расходы.

7.4 Реализация lock-free очереди

Использует два атомарных указателя: head и tail. Сложность связана с АВА-проблемой, решается счётчиками.

8 Пул потоков

8.1 Что такое пул потоков?

Набор заранее созданных потоков, которые выполняют переданные задачи (обычно через очередь). Позволяет избежать накладных расходов на создание/уничтожение потоков.

8.2 Реализация пула потоков на C++

Основные компоненты:

- Очередь задач (потокобезопасная).
- Вектор потоков-воркеров.
- Флаг остановки.

Пример:

```

class thread_pool {
    std::vector<std::thread> workers;
    std::queue<std::function<void()>> tasks;
    std::mutex queue_mutex;
    std::condition_variable cv;
    bool stop = false;
public:
    thread_pool(size_t threads) {
        for (size_t i = 0; i < threads; ++i) {
            workers.emplace_back([this] {
                while (true) {
                    std::function<void()> task;
                    {
                        std::unique_lock<std::mutex> lock(queue_mutex);

```

```

        cv.wait(lock, [this]{ return stop || !tasks.
            empty(); });
        if (stop && tasks.empty()) return;
        task = std::move(tasks.front());
        tasks.pop();
    }
    task();
}
});
}
}
~thread_pool() {
    {
        std::lock_guard<std::mutex> lock(queue_mutex);
        stop = true;
    }
    cv.notify_all();
    for (auto& t : workers) t.join();
}
template<typename F>
void enqueue(F&& f) {
    {
        std::lock_guard<std::mutex> lock(queue_mutex);
        tasks.emplace(std::forward<F>(f));
    }
    cv.notify_one();
}
};

```

Для получения результата можно использовать `std::packaged_task` и `std::future`.

8.3 Предотвращение конкуренции за очередь работ

Используются блокировки или lock-free очереди. Также полезно локальное «воровство задач» (work stealing).